Simulations & Tidyverse lab: Intermediate Version

PSYC 201A, Data Simulation Lab 1

Samantha Kefer

2026-10-07

Table of contents

Setup 1
A. Normal distribution (worked example) 2
B. Binomial data (accuracy) 6
C. Reaction times (RT): shifted lognormal 11
D. Two conditions: valid vs invalid cue 15

Goals: (1) Understand how normal, binomial, and lognormal distributions describe behavioral data (accuracy, RTs, individual variability). (2) Practice generating and visualizing synthetic datasets using tidyverse functions. (3) Learn how to write reproducible lab reports with inline statistics in Quarto.

Workflow: This intermediate version provides starter code and scaffolding to help you complete each task. Fill in the missing pieces and adapt the provided code.

Setup

If you finished getting-started-with-r, the tidyverse is already installed. If not, run this chunk once, by hand (green arrow at its top right). It is set to `eval: false` so it does not run every time you render.

```
install.packages("tidyverse")
```

`set.seed()` makes the random numbers come out the same every time you render, so the numbers you write about in your text do not change under you. Change the seed and every simulated value below will change.

```r
library(tidyverse)
set.seed(2025)
```

A. Normal distribution (worked example)

Here, let's try to simulate some values from a normal distribution with a mean of 0 and a SD of 1.

The `rnorm` function is used below here.

```r
# you can play around with these
n <- 300 # number of datapoints
mu <- 0 # mean of the distribution
sd_norm <- 1 # sd of the distribution

# rnorm is your simulation function here
# take a look at how it's used by typing `?rnorm` in the Console
## note that this function is taking the inputs you specified above -- it doesn't matter wha
norm_df <- tibble(sim_values = rnorm(n, mu, sd_norm))

# so you know, this code would do the same thing
# norm_df <- tibble(sim_values = rnorm(300, 0, 1))

# okay, check out the top of this dataframe
head(norm_df)
```

```
# A tibble: 6 x 1
  sim_values
       <dbl>
1     0.621
2     0.0356
3     0.773
4     1.27
5     0.371
6    -0.163
```

Calculate some basic summaries of this dataframe and print them out

```r
# TODO: Calculate mean, sd, and n for norm_df
# Hint: Use summarise() with mean(), sd(), and n()
# Start with this structure and fill in the missing parts:

norm_df_summary <- norm_df |>
  summarise(
    mean = mean(sim_values),  # Fill in the column name
    sd = sd(sim_values),      # Fill in the column name
    n = n()            # This one is already done!
  )

# Print the results
print(norm_df_summary)
```

```
# A tibble: 1 x 3
     mean    sd     n
    <dbl> <dbl> <int>
1 0.00433 0.987   300
```

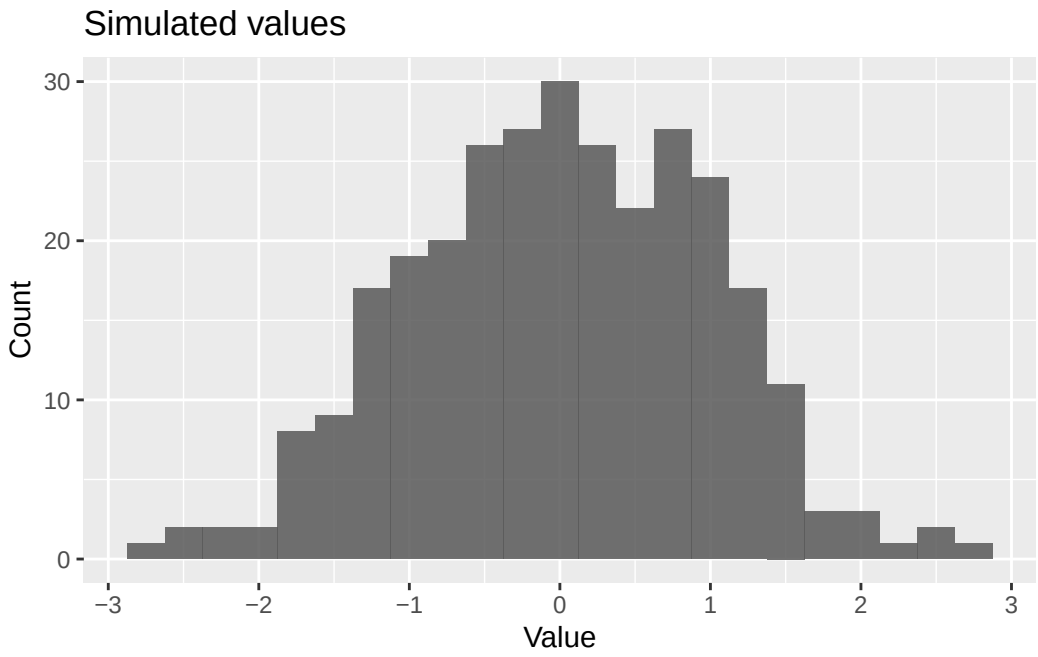
```r
# What happens if you increase the "n"? How do these summaries change?
```

I'm going to now plot a histogram of these values. We're going to spend a lot of time on figuring out how to plot things later – but here you can start learning the basics.

The first argument to the ggplot function is your dataframe – here, `norm_df`. The `aes` is where you specify what is on the x-axis, y-axis, colors, etc. These values come from the columns in `norm_df`.

`Alpha` is transparency. `binwidth` is how big the histogram bins are.

```r
ggplot(norm_df, aes(x=sim_values)) +
  geom_histogram(binwidth = 0.25, alpha = 0.85) +
  labs(title = "Simulated values", x = "Value", y = "Count")
```



Exercise: change `mu` and `sd_norm` to two other combos (e.g., 10 & 2; 10 & 5).

```
n <- 300
mu <- 10
sd_norm <- 2
norm_df2 <- tibble(sim_values = rnorm(n, mu, sd_norm))
norm_df2_summary <- norm_df2 |>
  summarise(
    mean = mean(sim_values),
    sd = sd(sim_values),
    n = n()
  )
print(norm_df2_summary)
```

```
# A tibble: 1 x 3
   mean    sd     n
  <dbl> <dbl> <int>
1  10.0  2.07   300
```

```
n <- 300
mu <- 10
sd_norm <- 5
```

```r
norm_df3 <- tibble(sim_values = rnorm(n, mu, sd_norm))
norm_df3_summary <- norm_df3 |>
  summarise(
    mean = mean(sim_values),
    sd = sd(sim_values),
    n = n()
  )
print(norm_df3_summary)
```

```
# A tibble: 1 x 3
   mean    sd     n
  <dbl> <dbl> <int>
1  10.0  4.87   300
```

Write one sentence with an inline mean (e.g., 0). Increasing mu from 0 to 5 dramatically increases sample mean from 0 to 10.02. Increasing sd_norm from 2 to 5 does not change sample mean but (intuitively) increases sd from 2.07 to 4.87.

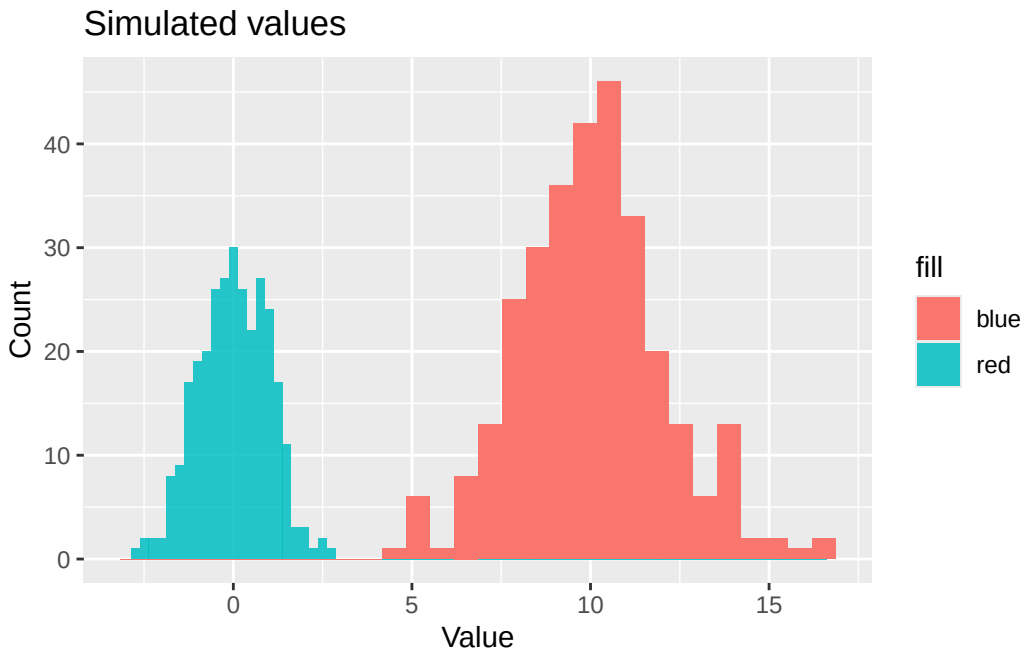
Bonus: Compute another kind of summary statistic on the dataset.

```r
norm_df_summary2 <- norm_df |>
  summarise(
    var = var(sim_values)
  )
print(norm_df_summary2)
```

```
# A tibble: 1 x 1
    var
  <dbl>
1 0.974
```

Bonus: Simulate another distribution and also plot it on the same histogram (you can make it a different color)

```r
ggplot(norm_df, aes(x=sim_values, fill = "red")) +
  geom_histogram(binwidth = 0.25, alpha = 0.85) +
  geom_histogram(data = norm_df2, aes(x=sim_values, fill="blue")) +
  labs(title = "Simulated values", x = "Value", y = "Count")
```



B. Binomial data (accuracy)

Okay, normally distributed data is great, but often not what we're working with in practice. Often, we have accuracies or reaction times.

First, we'll think about accuracy. To do so, we'll practice `rbinom()` and how to summarize it using some tidyverse functions. You should be able to adapt the code we just worked through but with a new function –

B1 — Bernoulli accuracy

Tasks:

- Simulate `n = 200` 0/1 outcomes with `p = 0.92` into `acc_df`.
- Compute **proportion correct** and a **count table** with `count()`.
- Plot overall proportion with `stat_summary()`.

```
# TODO: create a simulated dataframe using rbinom (remember, you can look at the help for the
# Hint: Use tibble() and rbinom(n=200, size=1, prob=0.92)
# Complete this line:
acc_df <- tibble(acc = rbinom(200, size = 1, prob = 0.92))
```

```r
# TODO: summarise and count
# Hint: Use summarise() to calculate proportion correct and n
# Complete this structure:
acc_df_summary <- acc_df |>
  summarise(
    prop_correct = mean(acc),  # Fill in the column name
    n = n()                    # This one is already done!
  )

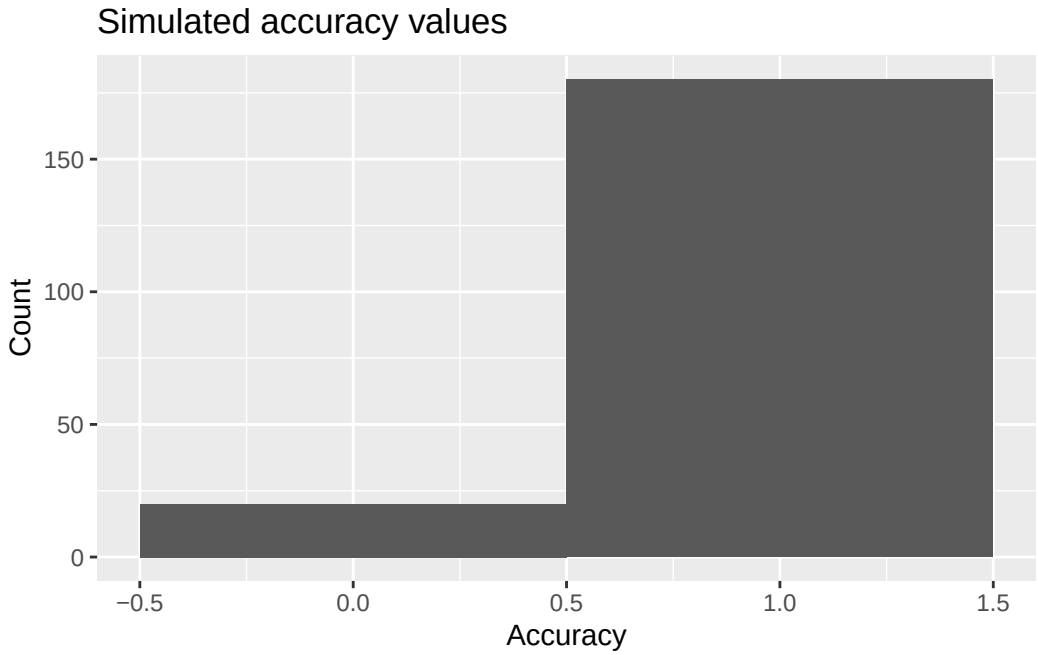
# Print the results
print(acc_df_summary)
```

```
# A tibble: 1 x 2
  prop_correct     n
         <dbl> <int>
1          0.9   200
```

```r
# Count table: how many 0s and 1s?
acc_df |> count(acc)
```

```
# A tibble: 2 x 2
    acc     n
  <int> <int>
1     0    20
2     1   180
```

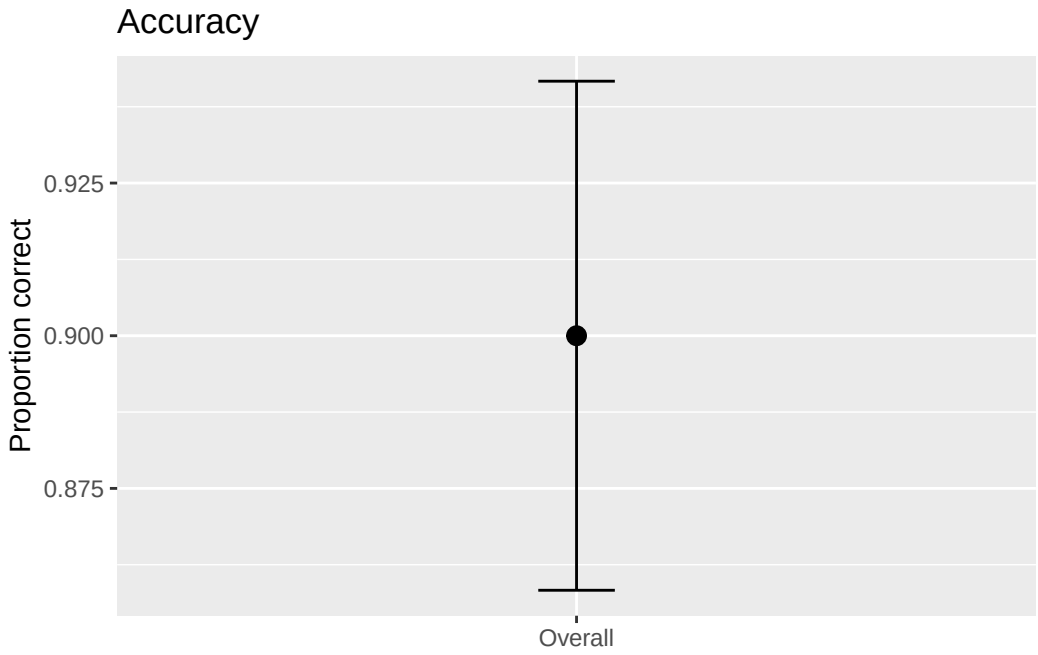
```r
## TO DO: Make a histogram of these values
# Hint: Use ggplot() with geom_histogram()
# Complete this structure:
ggplot(acc_df, aes(x = acc)) +  # Fill in dataframe and column name
  geom_histogram(binwidth = 1, alpha = 1) +  # Choose appropriate values
  labs(title = "Simulated accuracy values", x = "Accuracy", y = "Count")
```




```
# TODO: plot overall proportion
# Hint:  use `stat_summary(fun = mean)` for the mean

# TODO: Plot some measure of variability (SD or 95% CI)
# Hint: stat_summary(fun.data = mean_se, fun.args = list(mult = 1.96), geom = "errorbar", wi
# mean_se(mult = 1.96) gives the mean +/- 1.96 standard errors, an approximate 95% CI
# You have to give stat_summary an x-value, but since there isn't really one here you can ju

# Complete this structure:
ggplot(acc_df, aes(x = 'Overall', y = (acc))) +  # Fill in the column name
  stat_summary(fun = mean, geom = "point", size = 3) +
  stat_summary(fun.data = mean_se, fun.args = list(mult = 1.96), geom = "errorbar", width =
  labs(x = NULL, y = "Proportion correct", title = "Accuracy")
```



B2 — Two conditions + blocks (tidy)

Tasks:

- Build a tibble with `condition` in {easy, hard}, 100 trials each.
- Use different probabilities (e.g., 0.97 vs 0.87).
- Summarise `prop_correct` by `condition × block` and `pivot_wider()`.
- Bonus: Add a `participant` factor

```
# TODO: simulate
# Hint: Create a tibble with condition, block, and acc columns
# Use rep() to repeat conditions and blocks appropriately
# Use rbinom() with different probabilities for easy vs hard conditions

# Complete this structure:
acc2 <- tibble(
  condition = rep(c("easy", "hard"), each = 100),  # Fill in number of trials per condition
  block = rep(rep(1:4, each = 25), times = 2),   # Fill in trials per block
  acc = c(
    rbinom(100, 1, 0.97),  # Fill in number of trials for easy condition
    rbinom(100, 1, 0.87)   # Fill in number of trials for hard condition
  )
```

```
)

# Check per-block counts
acc2 |>
  add_count(block, name = "n_block") |>
  distinct(block, n_block)
```

```
# A tibble: 4 x 2
  block n_block
  <int>   <int>
1     1      50
2     2      50
3     3      50
4     4      50
```

```
# TODO: Summaries by condition × block
# Hint: Use group_by() and summarise()
# Complete this structure:
acc_by_cb <- acc2 |>
  group_by(condition, block) |>  # Fill in grouping variables
  summarise(
    prop_correct = mean(acc),  # Fill in column name
    trials = n(),
    .groups = "drop"
  )

# TODO: Pivot wider
# Hint: Use pivot_wider() to make conditions into separate columns
# Complete this structure:
acc_wide <- acc_by_cb |>
  pivot_wider(
    names_from = condition,     # Fill in the column to become column names
    values_from = prop_correct     # Fill in the column to become values
  )

# View the results
print("Summary by condition and block:")
```

```
[1] "Summary by condition and block:"
```

```
print(acc_by_cb)
```

```
# A tibble: 8 x 4
  condition block prop_correct trials
  <chr>     <int>        <dbl>  <int>
1 easy          1         1        25
2 easy          2         0.92     25
3 easy          3         1        25
4 easy          4         0.96     25
5 hard          1         0.8      25
6 hard          2         0.96     25
7 hard          3         0.88     25
8 hard          4         0.8      25
```

```
print("Wide format:")
```

```
[1] "Wide format:"
```

```
print(acc_wide)
```

```
# A tibble: 4 x 4
  block trials  easy  hard
  <int>  <int> <dbl> <dbl>
1     1     25  1     0.8
2     2     25  0.92  0.96
3     3     25  1     0.88
4     4     25  0.96  0.8
```

Inline prompt: "Overall accuracy = 0.9. Easy = 0.97, Hard = 0.86."

C. Reaction times (RT): shifted lognormal

RTs are famously right-skewed (ie., sometimes participants take a very long time to respond). This looks really different than a normal distribution. We'll use `t0 + LogNormal( , )` to approximately simulate their distribution. `t0` can be thought of a "baseline" reaction time that could vary by participant.

Simulating reaction times

Tasks:

- Simulate **n = 500** RTs with `t0 = 300`, `meanlog = log(250)`, `sdlog = 0.35` into `rt_df`.
- Plot a histogram and compute **mean**, **median**, **max**.
- Create `log_rt = log(rt)` and compare distributions

Here's a helper function to create lognormal RTs with a baseline shift. Note on these parameters: In a lognormal distribution, the parameters `meanlog` and `sdlog` describe the underlying normal distribution on the log scale, NOT the mean and standard deviation of the observed reaction times. The median of a lognormal is `exp(meanlog)`, so `meanlog` $= \log(250)$ makes the median of the lognormal part 250 ms. The helper adds `t0` on top, so with `t0 = 300` the median simulated RT is about $300 + 250 = 550$ ms. The `sdlog` parameter controls the amount of skew on the log scale: typical behavioral data are well-modeled by `sdlog` values between about 0.3 and 0.5. Using `sdlog` $= \log(20)$ would be incorrect, because `sdlog` already represents variability on the log scale; taking the log of a raw-scale SD would produce unrealistically long-tailed RTs spanning thousands of milliseconds. You don't need to understand all of these details, but I realize it could be confusing and so wanted to include this explanation.

```
# Helper: shifted lognormal RTs (t0 + LogNormal(meanlog, sdlog))
r_shifted_lognorm <- function(n, t0, meanlog, sdlog) {
  t0 + rlnorm(n, meanlog = meanlog, sdlog = sdlog)
}
```

```
# TODO: Simulate RTs and their logged RTs
# Hint: Use the r_shifted_lognorm() function with the specified parameters
# Create a tibble with rt and log_rt columns

# Complete this structure:
rt_df <- tibble(
  rt = r_shifted_lognorm(500, t0 = 300, meanlog = log(250), sdlog = 0.35)  # Fill in the para
) |>
  mutate(log_rt = log(rt))  # Fill in the column name

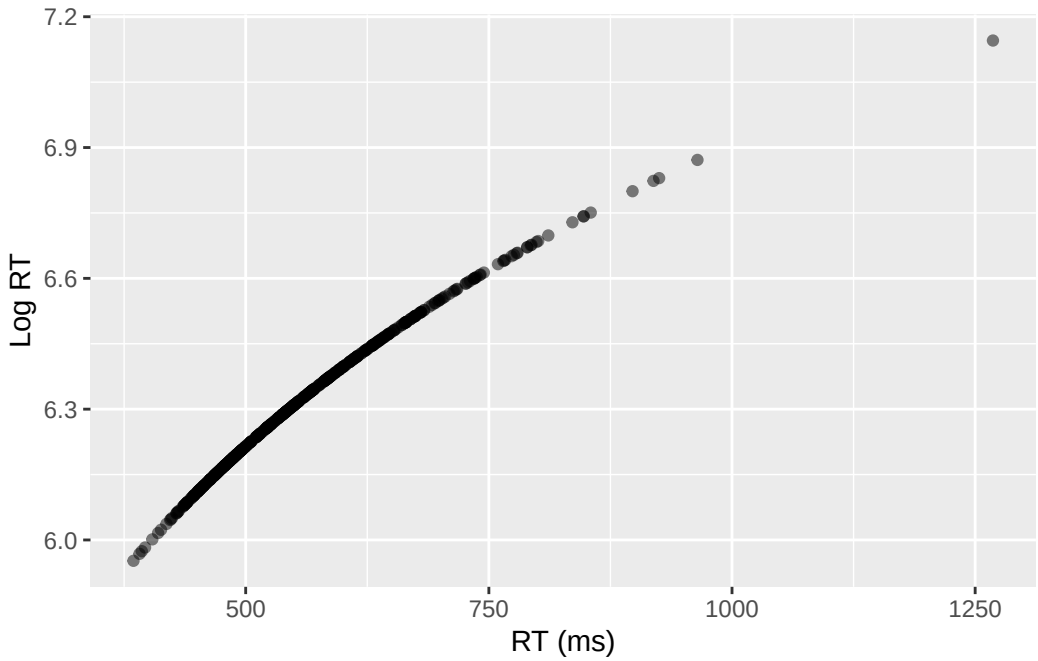
# Check the first few rows
head(rt_df)
```

```
# A tibble: 6 x 2
     rt log_rt
  <dbl>  <dbl>
1  473.   6.16
```

```
2  430.   6.06
3  476.   6.17
4  568.   6.34
5  574.   6.35
6  521.   6.26
```

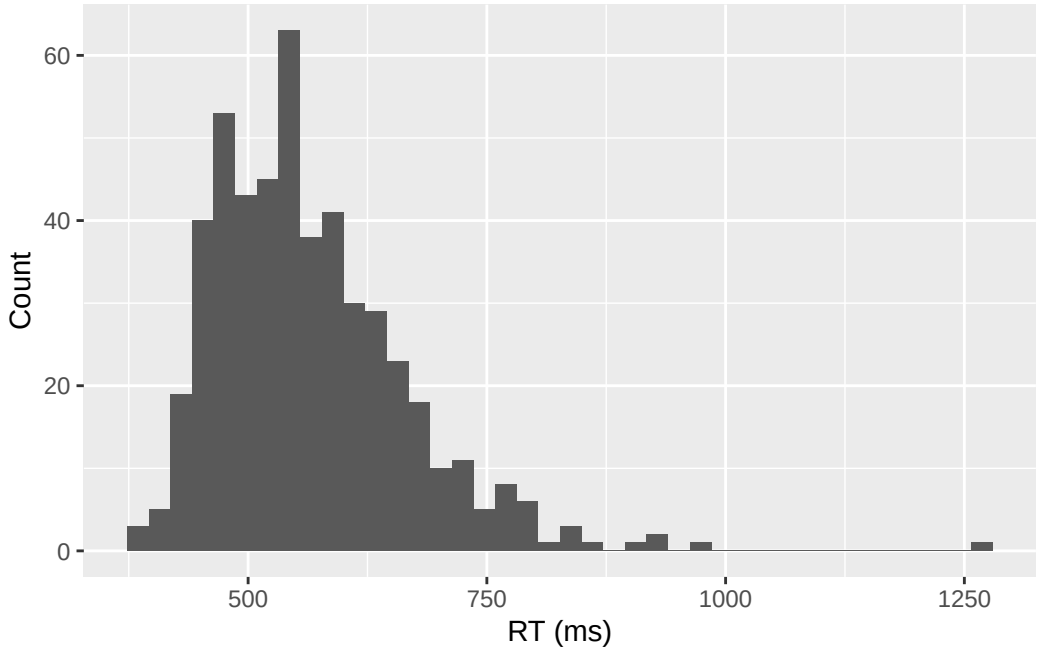
Now look at their relationship – use `ggplot()` with `geom_point()`, putting `rt` on the x-axis and `log_rt` on the y-axis.

```
# TODO: Plot rt vs log_rt
# Hint: ggplot() with geom_point()
ggplot(rt_df, aes(x = rt, y = log_rt)) +  # Fill in x and y column names
  geom_point(alpha = 0.5) +
  labs(x = "RT (ms)", y = "Log RT")
```

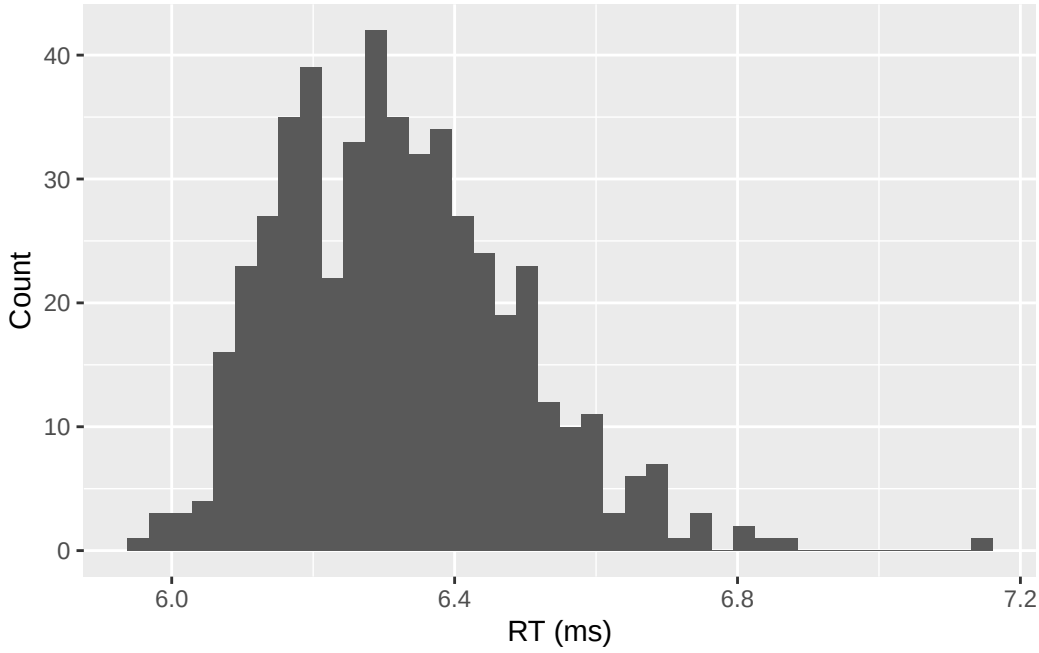


Now plot a histogram of `rt`, and another of `log_rt`. Which one looks more like a normal distribution?

```
# Complete this structure:
ggplot(rt_df, aes(x = rt)) +  # first rt, then copy the chunk and use log_rt
  geom_histogram(bins = 40) +
  labs(x = "RT (ms)", y = "Count")
```




```
ggplot(rt_df, aes(x = log_rt)) +  # first rt, then copy the chunk and use log_rt
  geom_histogram(bins = 40) +
  labs(x = "RT (ms)", y = "Count")
```



OK, now let's make some summaries. Use the summarize function to calculate the mean, median, max, and sd of the rt distribution. Note that you don't have to make a new dataframe for this.

```
# TODO: summaries
# Hint: Use summarise() to calculate mean_rt, median_rt, max_rt, sd_rt
# Complete this structure:
rt_df_summary <- rt_df |>
  summarise(
    mean_rt = mean(rt),    # Fill in column name
    median_rt = median(rt), # Fill in column name
    max_rt = max(rt),      # Fill in column name
    sd_rt = sd(rt)         # Fill in column name
  )

# Print the results
print(rt_df_summary)
```

```
# A tibble: 1 x 4
  mean_rt median_rt max_rt sd_rt
    <dbl>     <dbl>  <dbl> <dbl>
1    564.      548.  1268.  101.
```

D. Two conditions: valid vs invalid cue

Backstory. In a spatial attention task (think Posner cueing), each trial begins with a cue that is either valid (points to the upcoming target location) or invalid (points away). Valid cues help orient attention and typically speed responses; invalid cues slow responses. Reaction times (RTs) are right-skewed, as in the prior example. Let's try to simulate valid and invalid trials.

Add a **condition** factor. Suppose invalid cues slow responses by ~**40 ms** on average.

Try it

```
# TODO: Simulate RTs for valid and invalid conditions
# Hint: Use r_shifted_lognorm() with different parameters for each condition
# Invalid cues should be slower (higher meanlog)

# Complete this structure:
```

```r
rt_df_valid_invalid <- tibble(
  rt_valid = r_shifted_lognorm(500, t0 = 300, meanlog = log(250), sdlog = .35),     # Fill in
  rt_invalid = r_shifted_lognorm(500, t0 = 300, meanlog = log(290), sdlog = .35)   # Fill in
)

# Check the first few rows
head(rt_df_valid_invalid)
```

```
# A tibble: 6 x 2
  rt_valid rt_invalid
     <dbl>      <dbl>
1     623.       682.
2     559.       850.
3     578.       706.
4     471.       683.
5     549.       911.
6     453.       527.
```

Use `pivot_longer` to make this a long dataframe, where `rt` is just one column, and you have `condition` in another column. Use the `?` to look at the arguments for the function.

```r
# TODO: Use pivot_longer to reshape the data
# Hint: Convert rt_valid and rt_invalid columns into rt and condition columns
# Complete this structure:
rt_df_valid_invalid_longer <- rt_df_valid_invalid |>
  pivot_longer(
    cols = c(rt_valid, rt_invalid),        # Fill in the column names to pivot
    values_to = "rt",           # Fill in the name for the values column
    names_to = "condition"             # Fill in the name for the names column
  )

# Check the first few rows
head(rt_df_valid_invalid_longer)
```

```
# A tibble: 6 x 2
  condition     rt
  <chr>      <dbl>
1 rt_valid    623.
2 rt_invalid  682.
3 rt_valid    559.
4 rt_invalid  850.
```

```
5 rt_valid    578.
6 rt_invalid  706.
```

Now, use `group_by(condition)` before computing your summaries in tidyverse – you should see outputs grouped by condition.

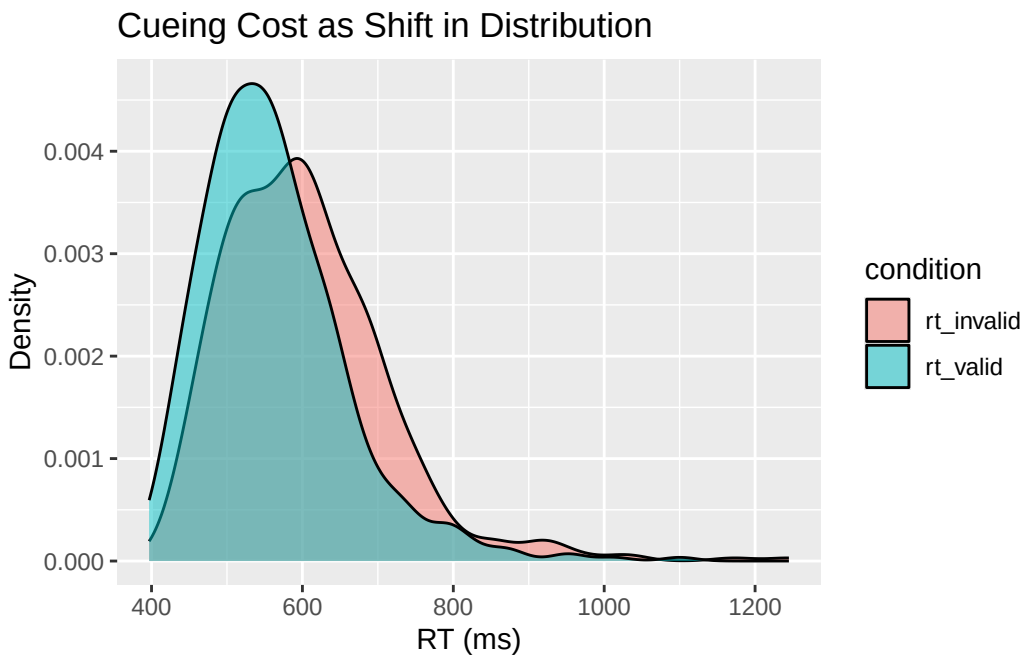
```r
# TODO: group_by() condition and summarise mean_rt, sd_rt
# Hint: Use group_by(condition) then summarise()
# Complete this structure:
valid_invalid_summary <- rt_df_valid_invalid_longer |>
  group_by(condition) |>  # Fill in the grouping variable
  summarise(
    mean_rt = mean(rt),  # Fill in column name
    sd_rt = sd(rt),      # Fill in column name
    n = n()
  )

# Print the results
print(valid_invalid_summary)
```

```
# A tibble: 2 x 4
  condition  mean_rt sd_rt     n
  <chr>        <dbl> <dbl> <int>
1 rt_invalid    604. 109.    500
2 rt_valid      562.  96.6   500
```

Now, make some sort of visual of the two distributions. Give the raw dataframe to ggplot, and use `geom_density` to plot the density (you can toggle `alpha` here to change the transparency). Use `fill=condition` within `aes` to specify that the two conditions should have different colors.

```r
## TODO: Use ggplot to make density plot of simulated data
# Hint: Use geom_density() with fill=condition in aes()
# Complete this structure:
ggplot(rt_df_valid_invalid_longer, aes(x = rt, fill = condition)) +  # Fill in dataframe and
  geom_density(alpha = 0.5) +            # Fill in alpha value (0-1)
  labs(title = "Cueing Cost as Shift in Distribution", x = "RT (ms)", y = "Density")
```



BONUS. Okay, so far you've been simulating "raw" data – i.e., 500 trials of valid, and 500 trials of invalid cueing. Try simulating 20 different participants, where each participant completes 200 trials (100 valid, 100 invalid).

```
# TODO: Simulate multiple participants
# Hint: Use a for loop to iterate over participants
# Each participant should have different baseline shifts
# Use bind_rows() to combine all participant data

# Set up parameters
num_participants <- 20
num_trials_per_condition <- 100

# Start with an empty tibble
df_experiment <- tibble()

# Loop over participants
for (i in 1:num_participants) {
  # Random baseline shift for this participant
  baseline_shift <- rnorm(1, mean = 0, sd = 80)

  # Simulate trials for this participant
  sim_subj <- tibble(
```

```
    participant = i,
    condition = rep(c("valid", "invalid"), each = num_trials_per_condition),
    trial = rep(1:num_trials_per_condition, times = 2),
    rt = c(
      r_shifted_lognorm(num_trials_per_condition, t0 = 300 + baseline_shift, meanlog = log(25
      r_shifted_lognorm(num_trials_per_condition, t0 = 300 + baseline_shift, meanlog = log(29
    )
  ) |>
    mutate(log_rt = log(rt))

  # Append to growing data frame
  df_experiment <- bind_rows(df_experiment, sim_subj)
}

# Check the structure
head(df_experiment)
```

```
# A tibble: 6 x 5
  participant condition trial    rt log_rt
        <int> <chr>     <int> <dbl>  <dbl>
1           1 valid         1  355.   5.87
2           1 valid         2  409.   6.01
3           1 valid         3  380.   5.94
4           1 valid         4  350.   5.86
5           1 valid         5  453.   6.12
6           1 valid         6  420.   6.04
```

```
# TODO: Summarize by participant and condition
# Hint: Use group_by(participant, condition) and summarise()
# Complete this structure:
df_experiment_summary <- df_experiment |>
  group_by(participant, condition) |>  # Fill in grouping variables
  summarise(
    mean_rt = mean(rt),  # Fill in column name
    n_trials = n(),
    .groups = "drop"
  )

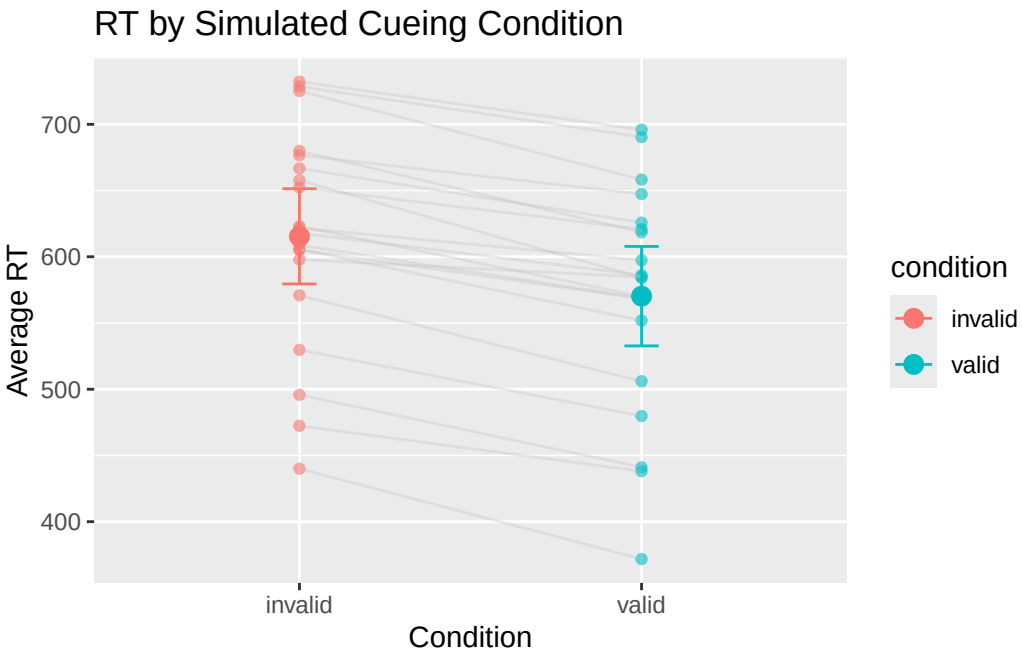
# Print the results
print(df_experiment_summary)
```

```
# A tibble: 40 x 4
```

```
   participant condition mean_rt n_trials
         <int> <chr>       <dbl>    <int>
 1           1 invalid      496.      100
 2           1 valid        441.      100
 3           2 invalid      680.      100
 4           2 valid        618.      100
 5           3 invalid      605.      100
 6           3 valid        568.      100
 7           4 invalid      725.      100
 8           4 valid        658.      100
 9           5 invalid      571.      100
10           5 valid        506.      100
# i 30 more rows
```

BONUS: Plot average RTs and 95 CIs over individual subjects. You can reuse `stat_summary` arguments for the mean and CIs from above. Plot raw data points from each participant using `geom_point`.

```
# TODO: Plot average RTs and 95 CIs over individual subjects
# Hint: Use stat_summary() for means and CIs, geom_point() for individual data
# Complete this structure:
ggplot(df_experiment_summary, aes(x = condition, y = mean_rt, color = condition, fill = condi
  geom_point(alpha = 0.6) +  # Individual subject means
  geom_line(aes(group = participant), alpha = 0.3, color = 'grey') +  # Connect data from eac
  stat_summary(fun = mean, geom = "point", size = 3) +  # Overall mean
  stat_summary(fun.data = mean_se, fun.args = list(mult = 1.96), geom = "errorbar", width = 0
  labs(x = "Condition", y = "Average RT", title = "RT by Simulated Cueing Condition")
```


RT by Simulated Cueing Condition
Average RT
700
600
500
400
invalid
valid
Condition
condition
invalid
valid